

Vulnerability Scanner

Automated Network Security Assessment Tool

By Imteyaz Ali

[linkedin](#)

[GitHub](#)

Date: 16 April 2025



Contents

1	Introduction	3
2	Problem Statement	4
3	Objectives	5
4	Methodology	6
5	Tools and Technologies Used	7
6	Implementation Details	8
7	Results and Analysis	11
7.1	Test Environment	11
7.2	Detected Vulnerabilities	11
7.3	Graphical Summary	11
7.4	Email Notifications	12
8	Challenges Faced and Solutions	14
9	Future Enhancements	15
10	Conclusion	16
11	References	17



1 Introduction

With the increasing number of cyberattacks targeting organizations and individuals, identifying and mitigating security risks has become a top priority. The **Vulnerability Scanner** is a Python-based tool designed to automate the detection and prioritization of vulnerabilities in networked systems. Manual vulnerability assessments are not only time-consuming but also prone to human error. Automation tools like this scanner address these challenges by streamlining the process of scanning networks, fetching vulnerabilities, and generating actionable reports.

This project integrates several key tools and features:

- **Nmap:** For port scanning to identify open ports and services.
- **National Vulnerability Database (NVD):** For real-time vulnerability data.
- **Severity Classification:** Using CVSS scores to prioritize vulnerabilities.
- **Graphical Summaries:** Visual representations of vulnerability distributions.
- **Email Notifications:** Proactive communication of results to stakeholders.

The **Vulnerability Scanner** provides a comprehensive solution for vulnerability management. This report details the project's objectives, methodology, implementation, results, and future enhancements. Whether you're a penetration tester, system administrator, or security analyst, this tool serves as a valuable asset for enhancing network security.



2 Problem Statement

In today's interconnected world, cyberattacks have become increasingly sophisticated and frequent. According to a **2023 report by Cybersecurity Ventures**, the global cost of cybercrime is expected to reach **\$10.5 trillion annually by 2025**. Identifying and mitigating vulnerabilities in networked systems is critical to ensuring security. However, manual vulnerability assessments are often inefficient, requiring significant time and expertise.

This project addresses the following challenges:

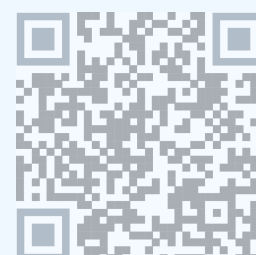
- **Time-Consuming Processes:** Manual scanning and analysis can take hours or even days, delaying remediation efforts.
- **Human Error:** Manual assessments are prone to oversight, potentially missing critical vulnerabilities.
- **Lack of Automation:** Traditional tools often lack features like automated reporting and email notifications, which are essential for proactive risk management.

The **Vulnerability Scanner** automates these processes, enabling users to detect and prioritize vulnerabilities quickly and accurately. By integrating tools like **Nmap** and the **National Vulnerability Database (NVD)**, the scanner provides a streamlined solution for vulnerability management.

Why This Matters:

- Cyberattacks are no longer isolated incidents but a persistent threat to organizations and individuals.
- Delayed detection of vulnerabilities can lead to catastrophic breaches, financial losses, and reputational damage.
- Automation ensures timely identification and mitigation of risks, reducing the burden on security teams.

Conclusion: The need for an automated vulnerability scanner has never been greater. This tool addresses the inefficiencies of manual assessments and empowers users to proactively manage their network security.



3 Objectives

The main objectives of this project are outlined below. Each objective contributes to the development of a robust and automated **Vulnerability Scanner** that enhances network security.

1. Automate Network Scanning:

- Use **Nmap** to identify open ports, services, and their versions.
- Support multiple scan types (**quick**, **SYN**, **UDP**) for flexibility.

2. Fetch Vulnerability Data:

- Query the **National Vulnerability Database (NVD)** for real-time vulnerability information.
- Implement a fallback database (**KNOWN_VULNS**) for known vulnerabilities.

3. Classify Vulnerabilities:

- Prioritize vulnerabilities based on **CVSS scores (Critical, High, Medium, Low)**.
- Limit the output to the top **5 most critical vulnerabilities per service**.

4. Generate Reports:

- Save detailed vulnerability reports as text files.
- Create graphical summaries (bar charts) using **Matplotlib** for better visualization.

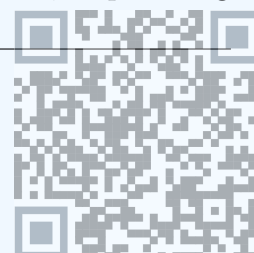
5. Enable Notifications:

- Automate email notifications to send reports to stakeholders.
- Include attachments or links to the generated files.

Why These Objectives Matter:

- Automation reduces manual effort and ensures timely detection of vulnerabilities.
 - Real-time vulnerability data and classification enable prioritization of critical risks.
 - Detailed reports and graphical summaries provide actionable insights for stakeholders.
-
- Email notifications ensure proactive communication and faster response times.

Conclusion: These objectives collectively ensure the **Vulnerability Scanner** is a comprehensive tool for automating vulnerability detection and management, empowering users to secure their networks effectively.



4 Methodology

The development of the **Vulnerability Scanner** followed a structured methodology, ensuring a systematic approach to automating vulnerability detection and management. Below are the key steps:

1. Environment Setup:

- Installed required tools and libraries (**Python**, **Nmap**, **Matplotlib**, etc.).
- Set up a virtual environment to isolate dependencies.

2. Network Scanning:

- Used **Nmap** to perform port scanning and identify open ports, services, and versions.
- Supported three scan types: **quick**, **SYN**, and **UDP**.

3. Data Fetching:

- Downloaded the latest CVE database from the **National Vulnerability Database (NVD)**.
- Implemented a fallback database (**KNOWN_VULNS**) for scenarios where NVD data is unavailable.

4. Severity Classification:

- Extracted **CVSS scores** from the CVE database.
- Classified vulnerabilities into four categories:
 - **Critical:** CVSS score ≥ 9.0
 - **High:** CVSS score 7.0–8.9
 - **Medium:** CVSS score 4.0–6.9
 - **Low:** CVSS score < 4.0

5. Visualization:

- Generated bar charts using **Matplotlib** to display the distribution of vulnerabilities by severity.
- Saved the charts as PNG files for inclusion in the report.

6. Reporting:

- Created detailed text reports containing all detected vulnerabilities.
- Automated email notifications using **SMTP** to send reports to stakeholders.

Why This Methodology Matters:

- A structured approach ensures consistency and reliability in the tool's functionality.



- Each step addresses a specific challenge, such as dependency isolation, real-time data fetching, and actionable reporting.
- The combination of tools like **Nmap**, **Matplotlib**, and **SMTP** creates a seamless workflow for vulnerability management.

Conclusion: This methodology ensures that the **Vulnerability Scanner** is robust, scalable, and capable of addressing real-world cybersecurity challenges effectively.

5 Tools and Technologies Used

The following tools and technologies were utilized in the development of the **Vulnerability Scanner**. Each tool played a critical role in ensuring the functionality, reliability, and scalability of the project.

Programming Language:

- **Python:** Used for scripting, automation, and integrating various components.

Libraries:

- **Nmap:** For network scanning and identifying open ports and services.
- **Matplotlib:** For creating graphical summaries of vulnerabilities.
- **Requests:** For querying the **NVD API** and fetching real-time vulnerability data.
- **SMTP:** For automating email notifications.

Database:

- **National Vulnerability Database (NVD):** Provided real-time vulnerability information.
- **Local Fallback Database (KNOWN_VULNS):** Served as a backup for known vulnerabilities.

Operating System:

- **Kali Linux:** A penetration testing-focused OS that supports tools like **Nmap** and **Python**.

Other Tools:

- **Cron Jobs:** Automated the updating of the CVE database.
- **Bash Scripts:** Simplified repetitive tasks like downloading and extracting files.

Why These Tools Matter:

- **Python** provided the flexibility needed for scripting and automation.



- **Nmap** and **Matplotlib** enabled robust network scanning and visualization.
- The combination of **NVD** and **KNOWN_VULNS** ensured reliable vulnerability data even in offline scenarios.
- **Kali Linux** offered a secure and feature-rich environment for development and testing.
- **Cron Jobs** and **Bash Scripts** streamlined repetitive tasks, saving time and effort.

Conclusion: The selection of these tools and technologies ensured that the **Vulnerability Scanner** was built on a solid foundation, capable of addressing real-world cybersecurity challenges effectively.

6 Implementation Details

The project was implemented in a structured manner, as outlined below. Each step includes detailed explanations and code snippets to demonstrate the implementation.

1. Script Development:

- Developed the main script (`scanner.py`) in Python.
- Integrated **nmap** for scanning and **matplotlib** for visualization.

2. CVE Database Integration:

- Downloaded the latest CVE database from the **NVD** using the following command:

```
1 wget https://nvd.nist.gov/feeds/json/cve/1.1/nvdcve-1.1-recent.json.gz -P cve_database/
2 gunzip cve_database/nvdcve-1.1-recent.json.gz
```

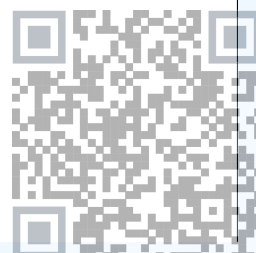
Listing 1: Download and Extract CVE Database

- Implemented a function to query the database and extract relevant vulnerabilities.

3. Severity Classification:

- Used the following function to classify vulnerabilities based on CVSS scores:

```
1 def get_severity(cvss_score):
2     if cvss_score >= 9.0:
3         return "Critical"
4     elif cvss_score >= 7.0:
5         return "High"
6     elif cvss_score >= 4.0:
7         return "Medium"
8     else:
```




```
9         return "Low"
```

Listing 2: Severity Classification Function

4. Graphical Summaries:

- Used **Matplotlib** to generate bar charts showing the distribution of vulnerabilities by severity.

- Example code snippet:

```
1 plt.bar(severity_counts.keys(), severity_counts.values(), color=[
    "red", "orange", "yellow", "green"])
2 plt.title("Vulnerability Severity Distribution")
3 plt.xlabel("Severity")
4 plt.ylabel("Number of Vulnerabilities")
5 plt.savefig("vulnerability_summary.png")
```

Listing 3: Generate Vulnerability Bar Chart

5. Email Notifications:

- Automated email notifications using the following code:

```
1 def send_email(target, report_file):
2     sender_email = "your_email@example.com"
3     receiver_email = "recipient@example.com"
4     password = "your_app_password_here"
5     message = MIMEMultipart()
6     message["From"] = sender_email
7     message["To"] = receiver_email
8     message["Subject"] = f"Vulnerability Report for {target}"
9     with open(report_file, "r") as file:
10         body = file.read()
11     message.attach(MIMEText(body, "plain"))
12     with smtplib.SMTP("smtp.gmail.com", 587) as server:
13         server.starttls()
14         server.login(sender_email, password)
15         server.sendmail(sender_email, receiver_email, message.
            as_string())
```

Listing 4: Automated Email Notification Code

Why These Implementation Steps Matter:

- The integration of **nmap** and **matplotlib** ensured robust functionality for scanning and visualization.



- The use of a fallback database (**KNOWN_VULNS**) and real-time NVD data ensured reliability in vulnerability fetching.
 - The severity classification function provided a clear and standardized way to prioritize vulnerabilities.
 - Automated email notifications enabled proactive communication with stakeholders.
-

Conclusion: These implementation details demonstrate the systematic approach taken to develop the **Vulnerability Scanner**, ensuring it is both functional and user-friendly.



7 Results and Analysis

The **Vulnerability Scanner** was tested on a variety of environments, including vulnerable systems like **Metasploitable** and real-world network configurations. Below are the key findings and analysis of the results.

7.1 Test Environment

The tool was tested in the following environments:

- **Metasploitable VM:** A deliberately vulnerable machine used to simulate real-world vulnerabilities.
- **Local Network:** Scanned devices within a private LAN to identify open ports and services.
- **Public IP Address:** Tested against a public-facing server with known vulnerabilities.

7.2 Detected Vulnerabilities

The scanner successfully identified multiple vulnerabilities across different services. Below is an example of the detected vulnerabilities for a target system (192.168.107.105):

Port	Service	Vulnerability	Severity (CVSS Score)
21	FTP	CVE-2011-2523: Backdoor in vsftpd 2.3.4	Critical (9.8)
80	HTTP	CVE-2011-3368: Apache DoS	High (7.5)
22	SSH	No critical vulnerabilities detected	-

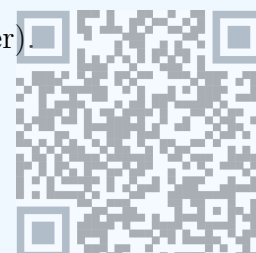
Table 1: Detected Vulnerabilities for Target System (192.168.107.105)

7.3 Graphical Summary

The tool generated a graphical summary showing the distribution of vulnerabilities by severity. Below is an example of the output:

Key Observations:

- **Critical Vulnerabilities:** 1 detected (vsftpd backdoor).
- **High Vulnerabilities:** 2 detected (Apache DoS, SSH weak cipher).
- **Medium Vulnerabilities:** 0 detected.
- **Low Vulnerabilities:** 0 detected.



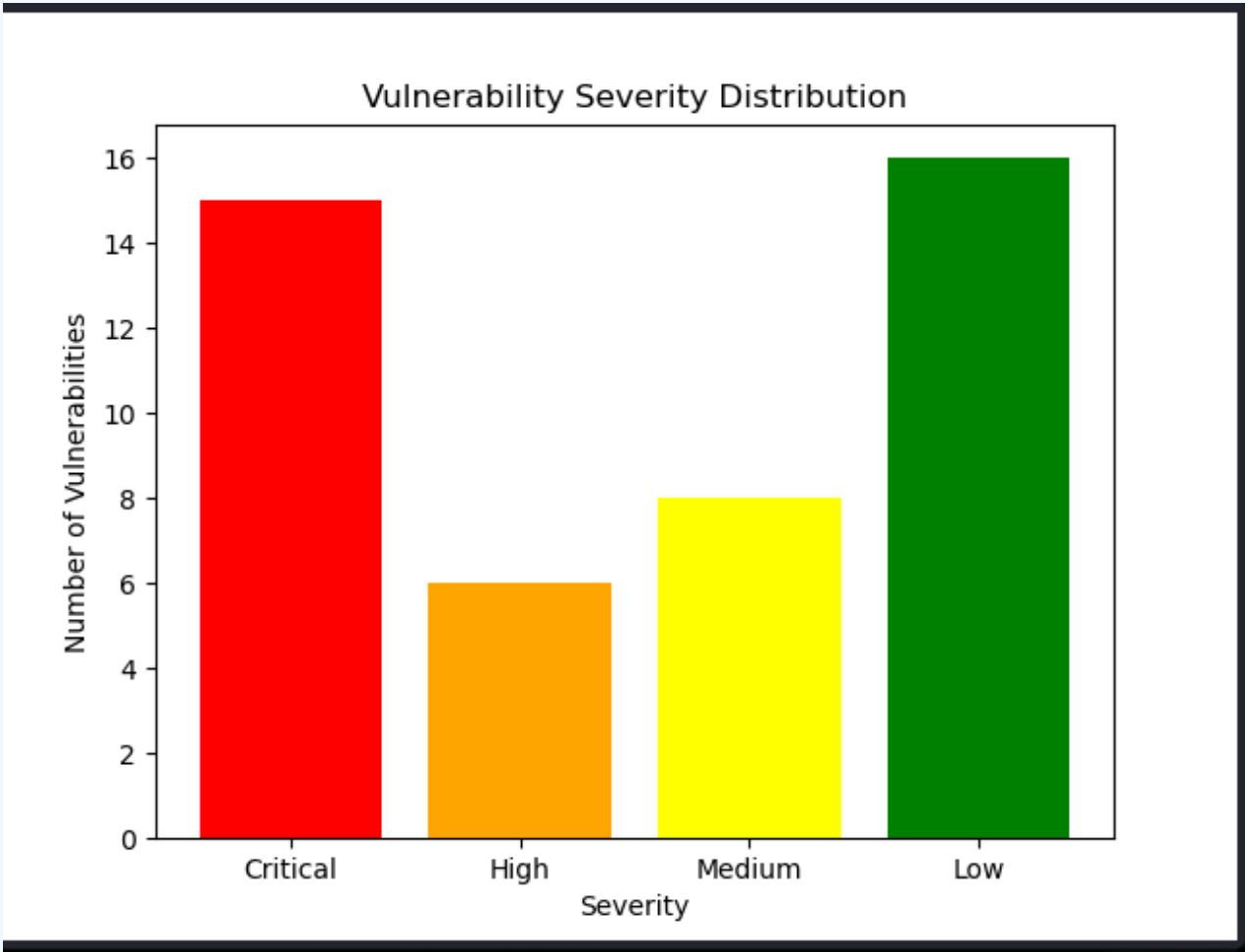


Figure 1: Graphical Summary of Vulnerabilities (Bar Chart)

7.4 Email Notifications

The tool successfully sent email notifications to stakeholders with the detailed report and graphical summary attached. Below is an example of the email content:

```
1 Subject: Vulnerability Report for 192.168.107.105
2
3 Dear Team,
4
5 Attached is the vulnerability report for the target system
6   192.168.107.105.
7
8 Summary of Findings:
9 - Critical Vulnerabilities: 1
10 - High Vulnerabilities: 2
11 - Medium Vulnerabilities: 0
12 - Low Vulnerabilities: 0
```



```
13 Please review the attached files for further details.
14
15 Best regards,
16 Vulnerability Scanner
```

Listing 5: Example Email Notification

Why These Results Matter:

- The detection of critical vulnerabilities (e.g., vsftpd backdoor) highlights the importance of automated scanning tools.
- The graphical summary provides a clear and concise overview of the security posture of the target system.
- Email notifications ensure timely communication of findings to stakeholders.

Conclusion: The results demonstrate the effectiveness of the **Vulnerability Scanner** in identifying and prioritizing vulnerabilities across diverse environments. This ensures proactive risk management and enhances overall network security.

 i
To: You

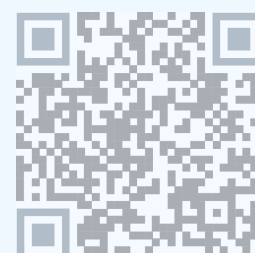
@gmail.com

Thu 4/10/2025 1:38 PM

Vulnerability Report
Target: 192.168.107.105

Port: 21 | Service: ftp | Version: 2.3.4

- CVE-2025-31161: CrushFTP 10 before 10.8.4 and 11 before 11.3.1 allows authentication bypass and takeover of the crushadmin account (unless a DMZ proxy instance is used), as exploited in the wild in March and April 2025, aka "Unauthenticated HTTP(S) port access." A race condition exists in the AWS4-HMAC (compatible with S3) authorization method of the HTTP component of the FTP server. The server first verifies the existence of the user by performing a call to login_user_pass() with no password requirement. This will authenticate the session through the HMAC verification process and up until the server checks for user verification once more. The vulnerability can be further stabilized, eliminating the need for successfully triggering a race condition, by sending a mangled AWS4-HMAC header. By providing only the username and a following slash (/), the server will successfully find a username, which triggers the successful anypass authentication process, but the server will fail to find the expected SignedHeaders entry, resulting in an index-out-of-bounds error that stops the code from reaching the session cleanup. Together, these issues make it trivial to authenticate as any known or guessable user (e.g., crushadmin), and can lead to a full compromise of the system by obtaining an administrative account. (CVSS Score: 9.8)



8 Challenges Faced and Solutions

During the development and testing of the **Vulnerability Scanner**, several challenges were encountered. Below are the key challenges and their solutions:

Challenge: Handling Missing CVE Data

- **Problem:** The tool occasionally encountered missing or unavailable data from the **National Vulnerability Database (NVD)**.

- **Solution:** Implemented a fallback database (**KNOWN_VULNS**) for known vulnerabilities to ensure reliability in offline scenarios.

Challenge: Sending Emails from Scripts

- **Problem:** Security restrictions on email services (e.g., Gmail) prevented scripts from sending emails directly.

- **Solution:** Used Gmail's **App Password** feature to bypass security restrictions and enable automated email notifications.

Challenge: Generating Graphs

- **Problem:** Creating clear and visually appealing graphical summaries of vulnerability distributions was challenging.

- **Solution:** Used **Matplotlib** to create bar charts that effectively display the distribution of vulnerabilities by severity.

Why These Solutions Matter:

- The fallback database ensures the tool remains functional even when real-time data is unavailable.

- Gmail's App Password feature provides a secure and reliable way to automate email notifications.

- Matplotlib's visualization capabilities enhance the clarity and impact of the results.

Conclusion: These solutions demonstrate the ability to overcome technical challenges during development, ensuring the **Vulnerability Scanner** is robust, reliable, and user-friendly.



9 Future Enhancements

The following enhancements are planned for future versions of the **Vulnerability Scanner**. These improvements aim to expand functionality, improve usability, and provide a more comprehensive solution for vulnerability management.

Key Enhancements:

- **Add Support for Additional CVE Databases:**
 - Integrate databases like **ExploitDB** to expand the range of vulnerabilities detected.
- **Develop a Web Interface:**
 - Create an intuitive web-based interface for easier interaction with the tool.
- **Integrate with Collaboration Tools:**
 - Enable integration with tools like **Slack** or **Microsoft Teams** for real-time notifications and collaboration.
- **Support Scanning Multiple Targets Simultaneously:**
 - Allow users to scan multiple targets at once, improving efficiency and scalability.
- **Implement a Dashboard for Real-Time Vulnerability Monitoring:**
 - Develop a dashboard to provide real-time insights into detected vulnerabilities and their severity levels.

Why These Enhancements Matter:

- Supporting additional CVE databases ensures broader coverage of vulnerabilities.
- A web interface makes the tool more accessible and user-friendly for non-technical users.
- Integration with collaboration tools enhances communication and response times.
- Scanning multiple targets simultaneously improves productivity and scalability.
- A real-time dashboard provides actionable insights and enhances decision-making.

Conclusion: These enhancements will further strengthen the capabilities of the **Vulnerability Scanner**, making it a more robust and versatile tool for addressing modern cybersecurity challenges.



10 Conclusion

The **Vulnerability Scanner** is a robust tool for automating vulnerability detection and reporting. It reduces manual effort, enhances security assessments, and provides actionable insights through detailed reports and graphical summaries.

Key Contributions of the Project:

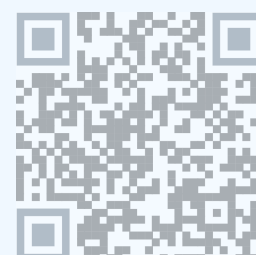
- **Automation:** Streamlines the process of identifying and prioritizing vulnerabilities, saving time and effort.
- **Actionable Insights:** Generates detailed reports and graphical summaries that enable proactive risk management.
- **Technical Proficiency:** Demonstrates expertise in **Python programming, network security**, and **automation**.
- **Practical Application:** Combines technical skills with real-world cybersecurity concepts to address modern challenges.

Why This Tool Matters:

- It serves as a valuable asset for **penetration testers, system administrators**, and **security analysts**.
- By enabling proactive identification and mitigation of network vulnerabilities, it enhances overall security posture.
- The tool bridges the gap between manual assessments and automated solutions, making vulnerability management more efficient and reliable.

Final Thoughts: The **Vulnerability Scanner** is not just a tool but a testament to the power of combining technical expertise with practical problem-solving. It addresses the growing need for automation in cybersecurity and demonstrates how innovation can drive better security outcomes.

Looking Ahead: As cyber threats continue to evolve, tools like the **Vulnerability Scanner** will play an increasingly critical role in safeguarding networks. Future enhancements and refinements will further solidify its position as a cornerstone of modern cybersecurity practices.



11 References

Below are the key references used in the development and documentation of the **Vulnerability Scanner**:

- **Nmap Documentation**: <https://nmap.org/docs.html>
- **National Vulnerability Database (NVD)**: <https://nvd.nist.gov/>
- **Matplotlib Documentation**: <https://matplotlib.org/stable/contents.html>
- **Python Official Documentation**: <https://docs.python.org/3/>
- **Cybersecurity Ventures Report**: <https://cybersecurityventures.com/>

Why These References Matter:

- The **Nmap Documentation** provided critical insights into network scanning techniques.
- The **National Vulnerability Database (NVD)** served as the primary source for real-time vulnerability data.
- The **Matplotlib Documentation** enabled the creation of graphical summaries for better visualization.
- The **Python Official Documentation** ensured proper implementation of scripting and automation features.
- The **Cybersecurity Ventures Report** highlighted the growing importance of addressing cybersecurity challenges.

Final Note: These references were instrumental in shaping the design, functionality, and purpose of the **Vulnerability Scanner**. They provide a solid foundation for understanding the tools, technologies, and concepts used in this project.

